

掰开揉碎的TAAC Baseline:模型篇

模型架构

为什么需要NS Tokenizer

模型初始化的第一部分是NS_tokenzier(非序列编词器)的初始化。

NS_tokenzier作用是把用户特征、物品特征这两类不是序列的特征编码成为Token，当然这里的Token不是指语言模型中的一个词元，而是指Token输入模型之后的一个向量，那么为什么NS Token需要编码而序列的特征则不需要编码呢？原因是序列特征都是同构的，也就是序列特征的结构上是相同的，他们在语义空间上是一致的，因此天然就可以进行编码后转化为语义向量，而NS Token，User特征和Token特征都是异构的，特征表达的含义、特征的尺度都是不一样的，因此不能进入同一个空间进行计算，所以需要进行Tokenizer。

那么怎么进行Tokenizer呢？这时最容易想到的方法是，我直接User一个Token，Item一个Token不就好了吗。这样当然是可以的，但是就出现另外一个问题，这个Token的表达过于杂糅，混乱，就像一个中文字同时要表达很多种意思的时候，就连你也会分不起，比如一个豪字，你也很难说这个豪具体表达了什么。这就是为什么不用一个Token的原因。

因此我们可以想到另外一种Tokenizer方法，就是我们将相同的语义的特征放在一块表达为一个Token，这样每个Token表达的含义就比较确定了，这就是GroupTokenizer的方式。如下图为ns_group.json:

```
"user_ns_groups": {
  "U1": [1, 15],
  "U2": [48, 49, 89, 90, 91],
  "U3": [80],
  "U4": [51, 52, 53, 54, 86],
  "U5": [82, 92, 93],
  "U6": [50, 60, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109],
  "U7": [3, 4, 55, 56, 57, 58, 59, 62, 63, 64, 65, 66]
},

"item_ns_groups": {
  "I1": [11, 13],
  "I2": [5, 6, 7, 8, 12],
  "I3": [16, 81, 83, 84, 85],
  "I4": [9, 10]
}
}
```

则表示，User被分为七组，这七组每组变为一个Token共七个Token，Item 四组四个Token。后面讲到模型里面的时候会讲讲是怎么划分Token的，简单来说，大家可以认为一个Token是一个d_model的向量，我把这些特征的embedding拼在一起，然后通过ffn变为d_model维度就是一个Token了。

上面提到的GroupTokenizer的优劣势都在于无参数的，无参数就注定了Token这部分难以学习，并且Token数量是写死的，比如Group最多能做到的User Token数也就是User的feat数量，也就是一个feat一个Token，难以扩充到更细粒度，而RankMixer则提出了通过切分的方式进行Token：首先将所有特征embedding后的维度拼接在一起，然后按照制定的ns token数量进行切分，比如所有特征embedding后的特征拼接在一起是768维度，用户指定了64个NS Token，那么首先会讲这些768维度切分为（64，12），即64组，每组12维，然后通过FFN映射到d_model维度。当然RankMixer这么设计还有其他的原因，也是为了后面无参的TokenMixing进行服务，这样的Token方式天然让一个Token混入了多样的特征信息。

```
cat_emb = torch.cat(all_embs, dim=-1) # (B, total_emb_dim) 拼接

# 2. Pad if needed
if self._pad_size > 0: 填充
    cat_emb = F.pad(cat_emb, (0, self._pad_size)) # (B, padded_total_dim)

# 3. Split into num_ns_tokens chunks and project each 切分
chunks = cat_emb.split(self.chunk_dim, dim=-1) # list of (B, chunk_dim)
tokens = []
for chunk, proj in zip(chunks, self.token_projs): 映射
    tokens.append(F.silu(proj(chunk)).unsqueeze(1)) # (B, 1, d_model)
```

ok，Tokenizer部分就先讲这些，后续代码拆解到再说。

baseline中的NS Tokenier

代码里面给出了两种选择，一种是Group的，另外一种则是Rankmixer的方式，这两种Token的具体思想前文已经提到过了，user和item的tokenizer是要分开的，理由也是因为这两类特征是具有异构性的。RankMixe的代码前文也拆过了，因此继续后续结构的拆解。

```

if ns_tokenizer_type == 'group':
    # Original: one NS token per group
    self.user_ns_tokenizer = GroupNSTokenizer(
        feature_specs=user_int_feature_specs,
        groups=user_ns_groups,
        emb_dim=emb_dim,
        d_model=d_model,
        emb_skip_threshold=emb_skip_threshold,
    )
    num_user_ns = len(user_ns_groups)

    self.item_ns_tokenizer = GroupNSTokenizer(
        feature_specs=item_int_feature_specs,
        groups=item_ns_groups,
        emb_dim=emb_dim,
        d_model=d_model,
        emb_skip_threshold=emb_skip_threshold,
    )
    num_item_ns = len(item_ns_groups)
elif ns_tokenizer_type == 'rankmixer':
    # RankMixer paper style: all embeddings cat → split → project
    # 0 means auto: fall back to group count
    if user_ns_tokens <= 0:
        user_ns_tokens = len(user_ns_groups)
    if item_ns_tokens <= 0:
        item_ns_tokens = len(item_ns_groups)
    self.user_ns_tokenizer = RankMixerNSTokenizer(
        feature_specs=user_int_feature_specs,
        groups=user_ns_groups,
        emb_dim=emb_dim,
        d_model=d_model,
        num_ns_tokens=user_ns_tokens,
        emb_skip_threshold=emb_skip_threshold,
    )
    num_user_ns = user_ns_tokens

    self.item_ns_tokenizer = RankMixerNSTokenizer(
        feature_specs=item_int_feature_specs,
        groups=item_ns_groups,
        emb_dim=emb_dim,
        d_model=d_model,
        num_ns_tokens=item_ns_tokens,
        emb_skip_threshold=emb_skip_threshold,
    )
    num_item_ns = item_ns_tokens

```

接下来是dense特征的FFN层也是分别对user和item的dense特征使用FFN进行映射到d_model作为一个Token

```

# Item dense feature projection (if available)
self.has_item_dense = item_dense_dim > 0
if self.has_item_dense:
    self.item_dense_proj = nn.Sequential(
        nn.Linear(item_dense_dim, d_model),
        nn.LayerNorm(d_model),
    )

# Total NS token count
self.num_ns = (num_user_ns + (1 if self.has_user_dense else 0)
               + num_item_ns + (1 if self.has_item_dense else 0))

```

因此NS的实际Token数量是，用户NS token数+物品NS token数+1（只有用户有dense特征），因此大家在调整参数的时候需要考虑一下这个1，保证token数加一后能被model整除。这是后续

TokenMixing的内容了，不过这里我有想到Dense特征是不是也可以拆为多个Token呢？（仅思考，没尝试过）

之后就是计算RankMixer是否可用，full mode时需要用 $d_model // T$ （后面会讲），因此要保证 $d_model \% T = 0$

```
T = num_queries * self.num_sequences + self.num_ns
if rank_mixer_mode == 'full' and d_model % T != 0:
    valid_T_values = [t for t in range(1, d_model + 1) if d_model % t == 0]
    raise ValueError(
        f"d_model={d_model} must be divisible by T=num_queries*num_sequences+num_ns
        f"{num_queries}*{self.num_sequences}+{self.num_ns}={T}. "
        f"Valid T values for d_model={d_model}: {valid_T_values}"
    )
```

Seq Tokenizer部分

相比起NS tokenizer，序列的token化起来会更简单一些，就是把一个序列中的每个时间都的全部特征输入embedding之后进行拼接，然后通过一个FFN就能转为Token，可以看到首先是为每个特征创建一个对应的embedding然后拼接在一起，所以FFN的输入维度是， $vs * emb_dim$ ，输出是token维度 d_model

```
for domain in self.seq_domains:
    vs = seq_vocab_sizes[domain] embed化
    embs, idx_map, is_id = _make_seq_embs(vs)
    self._seq_embs[domain] = embs
    self._seq_emb_index[domain] = idx_map
    self._seq_is_id[domain] = is_id
    self._seq_vocab_sizes[domain] = vs
    self._seq_proj[domain] = nn.Sequential(
        nn.Linear(len(vs) * emb_dim, d_model),
        nn.LayerNorm(d_model), 拼接后降维
    )
```

因此，通过想要修改自定义emb，就是想每个特征使用自定义的特征维度，而不是所有特征使用统一维度，可以修改两处地方，1.在下图这个emb_dim出使用特征对应的emb_dim，比如修改vocab_sizes为二元组，(vs, emb_dim)

```
embs_raw = []
for vs in vocab_sizes:
    skip = int(vs) <= 0 or (emb_skip_threshold > 0 and int(vs) > emb_skip_threshold)
    if skip:
        embs_raw.append(None)
    else:
        embs_raw.append(nn.Embedding(int(vs) + 1, emb_dim, padding_idx=0))
module_list = nn.ModuleList([e for e in embs_raw if e is not None])
```

第二处就是 原本由于维度一致，所以乘以特征数就行，改动后可以变化加和的形式。

```
self._seq_proj[domain] = nn.Sequential(
    nn.Linear(len(vs) * emb_dim, d_model),
    nn.LayerNorm(d_model),
)
```

通过每个特征自定义embedding大小，可以省下一些显存，让模型emb部分整体更可控。也能让一些不重要的特征别把重要的特征给脏到了（重要特征和非重要特征使用一样的维度可能影响模型学习重要特征）。

时间桶编码

```
# ===== Time Interval Bucket Embedding (optional) =====
if num_time_buckets > 0:
    self.time_embedding = nn.Embedding(num_time_buckets, d_model, padding_idx=0)
```

这个其实就这么一点，也没什么好说的，这个就是在数据部分传进来的time_bucket,然后做成embedding，但是这里可以注意一下，它的输出形状是d_model原因是后续它要加到序列的token上作为时间偏移量。

查询生成模块

ok呀，也是来到我们模型的第一个模块了，SeqQueryGenerator，其主要作用是为序列生成查询的Q-token，简而言之就是为了建立NS Token和S Token的桥梁，是通过NS Token来找那些序列Token对当前任务的预测作用最大。

```
# ===== HyFormer Components =====
# MultiSeqQueryGenerator
self.query_generator = MultiSeqQueryGenerator(
    d_model=d_model,
    num_ns=self.num_ns,
    num_queries=num_queries,
    num_sequences=self.num_sequences,
    hidden_mult=hidden_mult,
)
```

num_queries表示生成的查询token数量，注意是一个域生成num_queries个q token，num_sequences就是序列条数，因此生成的q_token实际数量，等于 num_queries * num_sequences

初始化方面

```

global_info_dim = (num_ns + 1) * d_model

# LayerNorm on global_info to prevent gradient explosion from large-dim concat
self.global_info_norm = nn.LayerNorm(global_info_dim)

# Each sequence has N independent FFNs
self.query_ffns_per_seq = nn.ModuleList([
    nn.ModuleList([
        nn.Sequential(
            nn.Linear(global_info_dim, d_model * hidden_mult),
            nn.SiLU(),
            nn.Linear(d_model * hidden_mult, d_model),
            nn.LayerNorm(d_model),
        )
        for _ in range(num_queries)
    ])
    for _ in range(num_sequences)
])

```

global_info_norm是总的维度大小，上述初始化的意思是在进行将所有NS token进行展平，和pool后的序列进行拼接后进行压缩，每条序列就这么压缩num_queries每个q_token。

```

for i in range(self.num_sequences):
    # MeanPool(Seq_i)
    valid_mask = ~seq_padding_masks[i] # True = valid
    valid_mask_expanded = valid_mask.unsqueeze(-1).float() # (B, L_i, 1)
    seq_sum = (seq_tokens_list[i] * valid_mask_expanded).sum(dim=1) # (B, D)
    seq_count = valid_mask_expanded.sum(dim=1).clamp(min=1) # (B, 1)
    seq_pooled = seq_sum / seq_count # (B, D) 序列池化

    # GlobalInfo_i = Concat(NS_flat, seq_pooled_i)
    global_info = torch.cat([ns_flat, seq_pooled], dim=-1) # (B, (M+1)*D)
    global_info = self.global_info_norm(global_info) NS Token和池化后拼接拉平

    # Generate N query tokens
    queries = [ffn(global_info) for ffn in self.query_ffns_per_seq[i]]
    q_tokens = torch.stack(queries, dim=1) # (B, Nq, D) 压缩为 q token
    q_tokens_list.append(q_tokens)

```

这个模块的优化点主要在于池化方法，目前是mean pool的方式，失去了非常多的序列信息，再进行压缩就不是很准，作为query损失太大。因此如何在池化时就找到和问题最为接近的信息去压缩为q_token是一个优化点。

模型的核心block

在查询生成模块之后是该模型的核心block，核心block的作用就是进行计算了，就是将ns_token q_token s_token放在一起进行特征的交叉，这里是我认为模型可以优化的一点，特征的交叉太过于晚，过于依赖前面提到的q_token的效果，这一点也可能是该模型scaling失效的原因。


```

self.blocks = nn.ModuleList([
    MultiSeqHyFormerBlock(
        d_model=d_model,
        num_heads=num_heads,
        num_queries=num_queries,
        num_ns=self.num_ns,
        num_sequences=self.num_sequences,
        seq_encoder_type=seq_encoder_type,
        hidden_mult=hidden_mult,
        dropout=dropout_rate,
        top_k=seq_top_k,
        causal=seq_causal,
        rank_mixer_mode=rank_mixer_mode,
    )
    for _ in range(num_hyformer_blocks)
])

```

初始化

该核心在初始化阶段主要有三个模块：1.seq_encoders 序列编码器 2.cross_attns 交叉注意力 3.mixer RankMixer的TokenMixing模块。因此能否验证scaling law这三个模块的设计也很重要，先看整体计算过程

```

S = self.num_sequences
Nq = self.num_queries

# 1. Independent Sequence Evolution per sequence
next_seqs = []
next_masks = []
for i in range(S):
    rc = rope_cos_list[i] if rope_cos_list is not None else None
    rs = rope_sin_list[i] if rope_sin_list is not None else None
    result = self.seq_encoders[i](
        seq_tokens_list[i], seq_padding_masks[i],
        rope_cos=rc, rope_sin=rs,
    )
    next_seq_i, mask_i = result
    next_seqs.append(next_seq_i)  # 序列编码
    next_masks.append(mask_i)

# 2. Independent Query Decoding per sequence
decoded_qs = []
for i in range(S):
    rc = rope_cos_list[i] if rope_cos_list is not None else None
    rs = rope_sin_list[i] if rope_sin_list is not None else None
    decoded_q_i = self.cross_attns[i](
        q_tokens_list[i], next_seqs[i], next_masks[i],
        rope_cos=rc, rope_sin=rs,
    )
    decoded_qs.append(decoded_q_i)  # q_token和编码序列进行交叉解码

# 3. Token Fusion: concatenate all decoded_q + ns_tokens
combined = torch.cat(decoded_qs + [ns_tokens], dim=1)  # (B, Nq*S + Nns, D)

# 4. Query Boosting
boosted = self.mixer(combined)  # (B, Nq*S + Nns, D)

# 5. Split back into per-sequence Q and NS 使用解码结果和ns token进入mixer模块
next_q_list = []
offset = 0
for i in range(S):
    next_q_list.append(boosted[:, offset:offset + Nq, :])
    offset += Nq
next_ns = boosted[:, offset:, :]

return next_q_list, next_ns, next_seqs, next_masks

```

从流程上来看是通过这个模块来获取下一状态的隐状态表示，作为后续用于分类的logits。马上也一个一个部分的来看三个小模块的流程吧。

序列encoder

在序列的encoder方式上，给了三种：1.swiglu 这是一个纯线性无注意力的方案 2.标准transformer 3.为了长序列问题而实现的一个longerencoder

```
if encoder_type == 'swiglu':
    return SwiGLUEncoder(d_model, hidden_mult, dropout)
elif encoder_type == 'transformer':
    return TransformerEncoder(d_model, num_heads, hidden_mult, dropout)
elif encoder_type == 'longer':
    return LongerEncoder(d_model, num_heads, top_k, hidden_mult, dropout, cause)
else:
    raise ValueError(f"Unknown encoder type: {encoder_type}")
```

前两种就让我们跳过，直接来到第三个longer，因为这道题的长序列问题确实是蛮突出的，单个用户的序列总和是上千的，但是按照目前baseline的方法采用1k，没有充分利用上数据。longer本质是一种序列压缩的方式，用topk条seq去压缩整个序列，也就是用topk个seq作为query去计算整条seq的注意力结果。

```
q, new_mask, q_pos_indices = self._gather_top_k(x, key_padding_mask)  # topk序列

# 2. Pre-LN
q_normed = self.norm_q(q)
kv_normed = self.norm_kv(x)

# 3. Build Q-side RoPE cos/sin by gathering from global cos/sin at top_k positions
q_rope_cos = None
q_rope_sin = None
if rope_cos is not None and rope_sin is not None:
    # rope_cos: (1, L_max, head_dim), q_pos_indices: (B, top_k)
    head_dim = rope_cos.shape[2]
    # Expand to batch dimension
    cos_expanded = rope_cos.expand(B, -1, -1)  # (B, L_max, head_dim)
    sin_expanded = rope_sin.expand(B, -1, -1)
    idx = q_pos_indices.unsqueeze(-1).expand(-1, -1, head_dim)  # (B, top_k, head_dim)
    q_rope_cos = torch.gather(cos_expanded, 1, idx)  # (B, top_k, head_dim)
    q_rope_sin = torch.gather(sin_expanded, 1, idx)

# 4. Cross Attention (no causal mask since Q and K have different lengths)
attn_out, _ = self.attn(
    query=q_normed,
    key=kv_normed,
    value=kv_normed,
    key_padding_mask=key_padding_mask,  # Original (B, L) mask
    rope_cos=q_rope_cos,
    rope_sin=q_rope_sin,
    q_rope_cos=q_rope_cos,
    q_rope_sin=q_rope_sin,
)
out = q + attn_out  # Residual based on q
```

这里要说的一点在于，这个topk的选择，不是和NS token最接近关系最好的，而是最新的topk条，当然有的时候不一定最新的seq代表了用户的倾向吧，所以这个部分可以稍微调整，但是也可以简单拉高topk。


```

# == Self Attention mode (subsequent MultiSeqHyFormerBlocks) ==
new_mask = key_padding_mask

# Pre-LN (Q and KV share norm_q)
x_normed = self.norm_q(x)

# Causal mask
attn_mask = None
if self.causal:
    attn_mask = nn.Transformer.generate_square_subsequent_mask(
        L, device=x.device
    )

attn_out, _ = self.attn(
    query=x_normed,
    key=x_normed,
    value=x_normed,
    key_padding_mask=key_padding_mask,
    attn_mask=attn_mask,
    rope_cos=rope_cos,
    rope_sin=rope_sin,
)

```

另外一个小bug在于这个因果注意力这一块，因果注意力，是下三角矩阵，但是前文提到过，这个数据的组织实际上新的在上面，旧的在下面。。。

ok，序列encoder的部分差不多就这样了，接下来是交叉注意力部分

交叉注意力

```

# Independent cross-attention per sequence
self.cross_attns = nn.ModuleList([
    CrossAttention(
        d_model=d_model,
        num_heads=num_heads,
        dropout=dropout,
        ln_mode='pre'
    )
    for _ in range(num_sequences)
])

```

这个其实也没有太多可以讲，这个就是调用的torch的cross attn，主要作用是用于

q_token和编码后seq_token的交叉，目的是加入序列Token和NS Token的混合，从而找到和当前任务重要的特征。

从计算过程看，交叉注意后的qs被和ns token拼接在一起用于后续的mixer，因此可以认为是通过q token在编码历史序列中查找最相关的历史token

2. Independent Query Decoding per sequence

```
decoded_qs = []
for i in range(S):
    rc = rope_cos_list[i] if rope_cos_list is not None else None
    rs = rope_sin_list[i] if rope_sin_list is not None else None
    decoded_q_i = self.cross_attns[i](
        q_tokens_list[i], next_seqs[i], next_masks[i],
        rope_cos=rc, rope_sin=rs,
    )
    decoded_qs.append(decoded_q_i)
```

3. Token Fusion: concatenate all decoded_q + ns_tokens

```
combined = torch.cat(decoded_qs + [ns_tokens], dim=1) # (B, Nq*S + Nns, D)
```

4. Query Boosting

```
boosted = self.mixer(combined) # (B, Nq*S + Nns, D)
```

TokenMixing

讲这个RankMixerBlock的话，感觉和讲RankMixer这个论文差不多了，baseline里的操作就是来自这篇论文中核心提到的TokenMixing方法，这个操作是无参的，核心思路是让token之间的信息进行流动，然后用FFN来进行特征变化，实现Token之间的特征互换的效果，比起注意力省下来很多计算成本。

```
Q_split = Q.view(B, T, self.T, self.d_sub)
Q_rewired = Q_split.transpose(1, 2).contiguous()
Q_hat = Q_rewired.view(B, T, D)
```

核心操作就三步：

- 1.将维度进行切变 (B,T,D) 切分为 (B,T,T,D/T)
- 2.维度互换(B,T时间,Token,D/T)变为(B,Token,T时间,D/T)
- 3.还原为 (B,T,D)

这个大家可能需要直接去脑部一下数据的流动过程：应用知乎上的一个解释

假设三个输入Token的向量为（为清晰起见，用字母表示值）：

- Token 1: $x_1=[a_1,a_2,a_3,a_4,a_5,a_6]$
- Token 2: $x_2=[b_1,b_2,b_3,b_4,b_5,b_6]$
- Token 3: $x_3=[c_1,c_2,c_3,c_4,c_5,c_6]$

第一步：切分头

每个Token被均分为3个头（每个头2维）：

- Token1:
头1: $[a_1,a_2]$ 头2: $[a_3,a_4]$ 头3: $[a_5,a_6]$
- Token2:
头1: $[b_1,b_2]$ 头2: $[b_3,b_4]$ 头3: $[b_5,b_6]$
- Token3:
头1: $[c_1,c_2]$ 头2: $[c_3,c_4]$ 头3: $[c_5,c_6]$

第二步：按头重组拼接

- 对于头1：收集所有Token的头1 $\rightarrow [a_1,a_2,b_1,b_2,c_1,c_2]$ ，得到混合后的Token s_1
- 对于头2：收集所有Token的头2 $\rightarrow [a_3,a_4,b_3,b_4,c_3,c_4]$ ，得到 s_2
- 对于头3：收集所有Token的头3 $\rightarrow [a_5,a_6,b_5,b_6,c_5,c_6]$ ，得到 s_3

第三步：形成输出

输出矩阵 SS 由 s_1,s_2,s_3 按行堆叠：

第一个输出Token s_1 包含了所有输入Token的第一个子空间的信息（即每个Token的前两个元素）。

第二个输出Token s_2 包含了所有输入Token的第二个子空间的信息。

第三个输出Token s_3 包含了所有输入Token的第三个子空间的信息。

这样，每个输出Token都融合了全局信息，但仅限于同一个子空间。后续的Per-token FFN可以在这个混合后的表示上进一步建模更复杂的交互。

在TokenMixing之后就是正常的FFN流程，无参的TokenMixing显著降低了计算成本，但我认为也可能阻碍了scaling law。

然后就是输出头了

```
# 2. Embed each sequence domain (dynamic)
seq_tokens_list = []
seq_masks_list = []
for domain in self.seq_domains:
    tokens = self._embed_seq_domain(
        inputs.seq_data[domain],
        self._seq_embs[domain], self._seq_proj[domain],
        self._seq_is_id[domain], self._seq_emb_index[domain],
        inputs.seq_time_buckets[domain])
    seq_tokens_list.append(tokens)
    mask = self._make_padding_mask(inputs.seq_lens[domain], inputs
    seq_masks_list.append(mask)

# 3. Generate independent Q tokens per sequence via MultiSeqQueryG
q_tokens_list = self.query_generator(ns_tokens, seq_tokens_list, s

# 4. Dropout + MultiSeqHyFormerBlock stack + output projection
output = self._run_multi_seq_blocks(
    q_tokens_list, ns_tokens, seq_tokens_list, seq_masks_list,
    apply_dropout=self.training
)

# 5. Classifier
logits = self.clsfier(output) # (B, action_num)
return logits
```

输出头就没有太多可说了。

推理的过程基本一致，没有太多好说。

总结

总结来看，model.py的代码看似复杂，但是整体的流程其实比想象的更简单一些，这里做一些简单的梳理。

1. NS 和序列 分别Token化
2. 产生每个序列的查询q_token
3. 对序列进行编码seq_encoder
4. 交叉注意力使用q_token查询编码的seq向量
5. 进行seq_token和n_token的特征交叉 (TokenMixing)
6. 输出头预测